

A CASE-STUDY BASED REPORT
ON
SENTINELFLOW: MACHINE LEARNING BASED DDoS ATTACK
DETECTION USING LSTM AND AUTOENCODER ANOMALY DETECTION

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

A.Y.: 2026-27

by

Tej Pratap Singh

Roll No. 2520030476

B. Aryan

Roll No. 2520030343

Under the Esteemed Guidance of

Ms. Jyothsna Datti

Designation / Department: [TO BE CONFIRMED]

Course Code: 25SC2107E | Section 4 - CSE



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract.....	
Introduction.....	
Background.....	
Motivation for the Project.....	
Overview of the Work Carried Out.....	
Scope and Responsible-Use Statement.....	
Problem Statement.....	
Case-Study Scenario.....	
Central Case-Study Question.....	
Formal Problem Definition.....	
Objectives that Guide the Study.....	
Assumptions and Limitations.....	
Dataset and Exploratory Data Analysis.....	
About the Dataset.....	
Purpose of EDA.....	
Preparation Steps Actually Performed.....	
EDA Exploration - Implementation.....	
Summary of EDA Findings.....	
Algorithm Selection and Implementation.....	
Conclusion.....	
Signature of the Team Members.....	

ABSTRACT

Distributed Denial-of-Service (DDoS) attacks threaten the availability of networked services by generating traffic volumes or patterns that exhaust service or network resources. This case study documents the design, implementation, evaluation, and deployment of SentinelFlow, the actual machine-learning project developed for the Machine Learning course. The implementation uses a cleaned and balanced 66,550-record subset of the CICDDoS2019 03-11 traffic data and compares three complementary detection strategies: Logistic Regression as a supervised per-flow baseline, a Long Short-Term Memory (LSTM) network for ordered eight-flow sequences, and a dense Autoencoder trained only on BENIGN training records for reconstruction-error-based anomaly detection. The final model-ready representation contains 68 numerical network-flow features. Data preparation removes duplicate traffic records, excludes identifiers and raw metadata from the model input, removes constant features, handles invalid numeric values using training-only statistics, creates fixed train/validation/test partitions, and fits a StandardScaler on the training split only. The Logistic Regression model achieves 99.7826% test accuracy, 99.8876% precision, 99.6774% recall, and 99.7824% F1-score on 14,260 individual flows. The temporal LSTM evaluates 14,211 eight-flow sequences and achieves 99.8593% accuracy, 99.9294% precision, 99.7886% recall, and 99.8590% F1-score. The Autoencoder is intentionally different: it learns only BENIGN behavior and flags records whose reconstruction error exceeds a validation-selected threshold of 0.126525490859982. With the final threshold, it achieves 90.1262% accuracy and 99.6073% recall, detecting 7,102 of 7,130 DDoS test records while producing more false positives than the supervised models. The trained artifacts are integrated into a Flask REST API and a React/Vite dashboard that validates uploaded CSV files, applies the saved preprocessing pipeline, runs the available models, and generates CSV-specific prediction analytics. The case study therefore represents the complete project as implemented, including its methodological constraints: the dataset is a balanced subset rather than the full CICDDoS2019 corpus, the LSTM ordering is chronological within predefined splits rather than a strict future-time holdout, and the deployed application is a batch CSV analysis tool rather than a real-time packet-capture system.

Table 1. Project implementation snapshot

Item	Implemented Project Detail
Application name	SentinelFlow
Dataset	CICDDoS2019 03-11 reduced project dataset
Final records	66,550 (33,275 BENIGN + 33,275 DDoS)
Model features	68 numerical flow features
Models	Logistic Regression, temporal LSTM, Autoencoder
Backend	Flask REST API with TensorFlow / Scikit-learn artifacts
Frontend	React + Vite dashboard
Primary output	BENIGN/DDoS predictions, anomaly scores, model summaries and dynamic CSV analytics

1. INTRODUCTION

1.1 Background

DDoS attacks are availability attacks in which a target service, server, or network is subjected to traffic that exceeds its capacity to respond normally. From a machine-learning perspective, network-flow records provide a practical representation of this behavior because they summarize communication using measurable attributes such as ports, protocols, packet counts, flow duration, packet-length statistics, inter-arrival times, flags, throughput, active/idle periods, and directionality.

The project does not operate on raw packet payloads. Instead, it uses flow-level numerical attributes from CICDDoS2019. This makes the learning task a structured-data problem in which each record describes one observed network flow. The project first establishes a conventional supervised baseline using Logistic Regression, then adds an LSTM so that eight consecutive flows can be evaluated as an ordered sequence, and finally adds an Autoencoder that models BENIGN traffic without using attack examples during its own training.

These three models answer related but different questions. Logistic Regression asks whether one scaled flow resembles BENIGN or DDoS examples learned from labeled data. The LSTM asks whether an ordered window of flows exhibits a temporal pattern associated with DDoS behavior. The Autoencoder asks whether a flow can be reconstructed well by a network trained only on BENIGN traffic. Their roles are therefore complementary rather than interchangeable.

The work was completed as an end-to-end system rather than as a notebook-only experiment. The trained scaler, feature order, classifier, deep-learning models, anomaly threshold, metrics, and graphs are saved as reusable artifacts. A Flask backend loads those artifacts, while a React/Vite interface named SentinelFlow accepts compatible CSV files and presents prediction summaries and live analytics.

1.2 Motivation for the Project

A single high-performing classifier can demonstrate binary classification, but it does not by itself explore whether temporal context or behavior-based anomaly detection adds a different perspective. The motivation for this project was therefore to study DDoS detection through three levels of increasing modeling complexity while retaining one common dataset and preprocessing pipeline.

Logistic Regression was retained as a strong, computationally efficient baseline. The LSTM was introduced because flow records can form time-ordered sequences and recurrent models can learn dependencies across successive observations. The Autoencoder was included because anomaly detection can be formulated without training the reconstruction model on DDoS examples: normal traffic is compressed and reconstructed, and large reconstruction errors indicate deviation from the learned BENIGN pattern.

A further motivation was reproducibility and demonstration. Model development, EDA, saved artifacts, backend inference, and frontend visualization were kept in the same project structure so that the final outcome could be shown to faculty as both a machine-learning experiment and a working software system.

1.3 Overview of the Work Carried Out

The implemented work followed the sequence shown in Figure 1. Every stage in the diagram corresponds to a file, dataset, model, graph, or software component present in the final project.

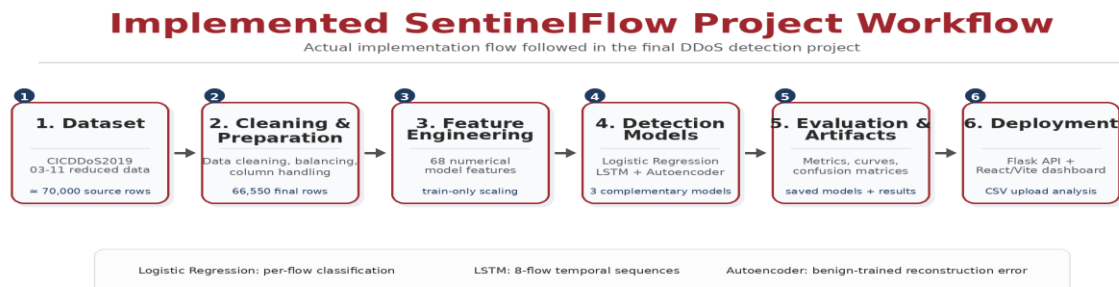


Figure 1. Project workflow derived from the implemented pipeline and deployment architecture

Figure 1. Implemented SentinelFlow project workflow reconstructed from the actual notebook, saved artifacts, backend, and frontend.

The project began with a user-provided 70,000-row reduction of CICDDoS2019 CSV-03-11. Duplicate-flow cleaning, balancing, invalid-value treatment, feature filtering, and split creation produced the final 66,550-record dataset. The model-ready file contains the 68 numerical predictors together with BinaryLabel, Label, and Split. A separate clean temporal file retains Timestamp and SourceCSV so that LSTM sequences can be ordered without feeding those metadata fields into the model.

The notebook then performs EDA, fits the training-only StandardScaler, trains the Logistic Regression model, constructs LSTM sequences, trains and evaluates the LSTM, trains the BENIGN-only Autoencoder, selects the final anomaly threshold from validation precision-recall behavior, saves all final metrics, and exports model artifacts. The backend reuses these artifacts instead of retraining models at application start.

The software layer exposes GET endpoints for health, metrics, and configuration, and POST endpoints for CSV validation and prediction. The frontend submits a CSV to the backend and displays both fixed research performance and dynamic analytics computed from the uploaded file. This separation prevents unlabeled live uploads from being incorrectly presented as if they had new measured accuracy values.

1.4 Scope and Responsible-Use Statement

The scope of SentinelFlow is defensive, academic DDoS traffic analysis using structured network-flow CSV data. The implemented system classifies or scores traffic; it does not generate DDoS traffic, exploit remote systems, scan external hosts, or automate offensive activity. Its purpose is to demonstrate preprocessing, supervised classification, temporal learning, anomaly detection, evaluation, and deployment in a controlled project setting.

Table 2. Scope and responsible-use boundaries

Within the implemented scope	Outside the implemented scope
Binary BENIGN vs DDoS analysis	Generation or execution of DDoS attacks
Batch CSV validation and inference	Live packet capture or inline firewall enforcement
Per-flow Logistic Regression and Autoencoder scoring	Automated blocking of hosts or network addresses
Eight-flow LSTM sequence prediction	Claim of guaranteed zero-day detection
Academic evaluation on the project dataset	Claim of production readiness on arbitrary networks

Because any statistical detector can produce false positives and false negatives, SentinelFlow should be treated as a decision-support demonstration rather than as an autonomous security authority. In a real operational setting, predictions would require validation against network context and human review. The project also avoids using Flow ID, Source IP, and Destination IP as predictive inputs, reducing dependence on direct identifiers in the learned models.

2. PROBLEM STATEMENT

The project addresses the problem of detecting whether network-flow traffic is BENIGN or DDoS while comparing three implemented modeling strategies under a common, leakage-aware preprocessing pipeline. The system must preserve the same feature order and scaling used during training so that saved models can be reused correctly during deployment.

2.1 Case-Study Scenario

Consider a network operations or security team that has exported a set of flow-level statistics from a monitored environment into a CSV file. The team needs a rapid indication of whether the traffic appears benign or contains DDoS-like behavior. The CSV contains the same numerical flow attributes expected by the trained project models. It may also contain SourceCSV and Timestamp metadata that can be used to order LSTM windows.

In the implemented SentinelFlow workflow, the operator uploads the CSV through the web dashboard. The Flask backend checks whether all 68 required model features are present. Invalid or non-numeric entries are converted to missing values and, for deployment inference, filled using the training means preserved in the fitted StandardScaler. The same scaler then transforms the input. Logistic Regression and the Autoencoder operate on every flow. If the LSTM model is available, the backend forms windows of eight flows, ordered by SourceCSV and Timestamp when both metadata columns are present, or by uploaded row order otherwise.

The dashboard returns model-specific attack rates, a primary BENIGN/DDoS distribution, Autoencoder reconstruction-error analytics, LSTM probability trends, and sample predictions. These are CSV-specific inference outputs. Separately, the dashboard can display the fixed evaluation metrics produced by the held-out project test set.

2.2 Central Case-Study Question

How effectively can a leakage-aware combination of per-flow Logistic Regression, eight-flow LSTM sequence modeling, and BENIGN-trained Autoencoder anomaly detection distinguish DDoS traffic in the project's CICDDoS2019 subset, and how can the trained artifacts be deployed consistently in a CSV-based web application?

2.3 Formal Problem Definition

Let x_i in $\mathbb{R}^{68}$ denote the numerical feature vector for flow i after the project's feature filtering, and let y_i in $\{0,1\}$ denote BinaryLabel, where 0 represents BENIGN and 1 represents DDoS. The supervised per-flow task is to learn a function $f_{LR}(x_i)$ that estimates the probability that $y_i = 1$.

For the temporal model, an LSTM input is an ordered window $S_t = [x_{(t-7)}, \dots, x_t]$ containing eight scaled flows from the same SourceCSV group. The sequence target is the BinaryLabel of the final flow in the window. The model $f_{LSTM}(S_t)$ returns a DDoS probability and uses 0.5 as the binary decision threshold during evaluation.

For the anomaly-detection task, the Autoencoder $g(\cdot)$ is trained only on BENIGN training vectors. For an input x , the anomaly score is the mean squared reconstruction error: $MSE(x, g(x)) = (1/68) * \sum_j (x_j - g(x)_j)^2$. The final project classifies the flow as anomalous/DDoS-like when this error exceeds the validation-selected threshold $\tau = 0.126525490859982$.

The evaluation objective is not to force the three models into one identical learning problem, but to compare their measured behavior while respecting their different units of analysis. Logistic Regression and the Autoencoder are evaluated on individual test flows; the LSTM is evaluated on eight-flow sequences and therefore has 14,211 test units rather than 14,260.

2.4 Objectives that Guide the Study

- Prepare a reproducible, balanced binary DDoS dataset from the project's 70,000-row CICDDoS2019 reduction without using identifiers or labels as predictive features.
- Perform EDA on class balance, attack-type composition, split sizes, important correlations, and representative feature distributions.
- Establish Logistic Regression as a supervised per-flow baseline using the common 68-feature representation.
- Construct chronological eight-flow sequences within each existing split and source CSV, and train an LSTM for temporal classification.
- Train an Autoencoder only on BENIGN training records and select the anomaly threshold using validation data rather than test labels.
- Evaluate the models using Accuracy, Precision, Recall, F1-score, ROC-AUC, PR-AUC, and confusion matrices.
- Save the fitted scaler, feature ordering, models, configuration, anomaly threshold, metrics, and graphs for deployment.
- Integrate the trained artifacts into a Flask API and React/Vite dashboard for compatible CSV analysis.
- Document methodological limitations and avoid presenting inference outputs or balanced-subset results as broader production guarantees.

2.5 Assumptions and Limitations

Table 3. Assumptions and limitations of the implemented case study

Assumption / Limitation	Impact on interpretation
Balanced subset rather than the full corpus	The 50:50 BENIGN/DDoS class balance is useful for controlled comparison but is not representative of every operational network.
Single project source day / archive subset	The final project is based on the CICDDoS2019 03-11 reduction; generalization to other days or datasets is not demonstrated here.
Binary target	All retained attack types are grouped into DDoS; the deployed models do not perform multiclass attack-family prediction.
LSTM split created before temporal ordering	Sequences are chronological within each predefined split, but the experiment is not a strict train-on-past/test-on-future holdout.
LSTM requires eight-flow context	The LSTM produces no sequence prediction for groups shorter than eight records.

Assumption / Limitation	Impact on interpretation
Batch CSV application	The implemented web system does not capture live packets or enforce network controls.
Autoencoder threshold dependence	Anomaly precision/recall changes with the selected reconstruction-error threshold; the final threshold is validation-selected.
Deployment fallback imputation	Training preparation uses training-only medians for invalid rate features, while the generic deployed API fills malformed/missing numeric inputs with StandardScaler training means before scaling.
Required schema	The backend rejects files missing any of the 68 required model features.
Upload limit	The Flask application sets a 64 MB CSV upload limit.

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 About the Dataset

The project uses a user-provided 70,000-row reduction of the CICDDoS2019 CSV-03-11 traffic data. The source reduction contains 91 columns and includes BENIGN traffic together with LDAP, MSSQL, NetBIOS, Portmap, Syn, UDP, and UDPLag attack labels. No other dataset is used for model training in the final project.

Cleaning removed 1,725 duplicate traffic records. The cleaning report records that all 1,725 removed duplicates belonged to BENIGN traffic. After deduplication, 68,275 unique records remained: 33,275 BENIGN records and 35,000 attack records. The attack side was then sampled to 33,275 records so that the binary dataset contained equal BENIGN and DDoS totals.

The resulting clean dataset contains 66,550 rows and 89 columns because temporal and source metadata are retained there for sequence construction. The model-ready dataset contains 71 columns: 68 numerical predictor features plus BinaryLabel, Label, and Split. Identifier and raw metadata fields are excluded from direct model input.

Table 4. Final traffic-label composition recorded by the cleaning report

Traffic label	Final records
BENIGN	33,275
MSSQL	6,850
Syn	5,594
Portmap	4,740
UDP	4,740
LDAP	4,739
NetBIOS	4,739
UDPLag	1,873

Table 5. Final binary-balanced train, validation, and test splits

Split	BENIGN	DDoS	Total
Train	21,391	21,391	42,782
Validation	4,754	4,754	9,508
Test	7,130	7,130	14,260

3.2 Purpose of EDA

EDA was used to verify that the cleaned data matched the intended binary problem before training and to identify characteristics that influence modeling decisions. In this project, EDA is not treated as a decorative step; the exported figures verify class balance, retained attack-family

composition, partition sizes, feature-target association, and relationships among important numerical predictors.

The EDA also supports implementation checks. For example, a perfectly balanced binary distribution confirms that the attack sampling step produced the intended final dataset. Split plots confirm the exact training, validation, and test sizes. Correlation analysis shows which flow properties are strongly associated with BinaryLabel, while histograms and boxplots expose skew and large ranges in rate and packet statistics, reinforcing the need for a consistent scaling pipeline.

3.3 Preparation Steps Actually Performed

1. Start from the 70,000-row, 91-column project reduction of CICDDoS2019 CSV-03-11.
2. Remove 1,725 duplicate traffic records, leaving 68,275 unique rows.
3. Retain all 33,275 unique BENIGN rows and sample the DDoS side from 35,000 attack rows down to 33,275 so that the binary classes are balanced.
4. Remove the redundant/index-style columns Unnamed: 0 and Fwd Header Length.1 from the clean data.
5. Identify invalid values in Flow Bytes/s and Flow Packets/s. The cleaning report records 1,618 invalid values in each of these two features before treatment.
6. Use training-only medians for these invalid numerical values during dataset preparation. The recorded medians are 432,091.8416259227 for Flow Bytes/s and 41,666.666666666666 for Flow Packets/s.
7. Exclude target, identifier, and raw metadata fields from model input: BinaryLabel, Label, Flow ID, Source IP, Destination IP, Timestamp, SourceCSV, Split, and SimillarHTTP.
8. Drop 12 constant features from the model representation, leaving 68 numerical model features.
9. Create the fixed train, validation, and test splits of 42,782, 9,508, and 14,260 flows respectively, with binary balance retained in every split.
10. Fit StandardScaler on the training feature matrix only and reuse it for validation, test, and saved-model deployment.
11. Preserve Timestamp and SourceCSV in the clean temporal dataset for LSTM ordering and grouping, but do not feed them as numerical model features.

Table 6. Constant features dropped during model feature preparation

Constant features removed from the model-ready representation	
Bwd Avg Bulk Rate	Bwd Avg Bytes/Bulk
Bwd Avg Packets/Bulk	Bwd PSH Flags
Bwd URG Flags	ECE Flag Count
FIN Flag Count	Fwd Avg Bulk Rate
Fwd Avg Bytes/Bulk	Fwd Avg Packets/Bulk
Fwd URG Flags	PSH Flag Count

3.4 EDA Exploration - Implementation

The EDA figures below are the actual exported project graphs stored under results/eda/. They are reproduced here without changing their underlying data.

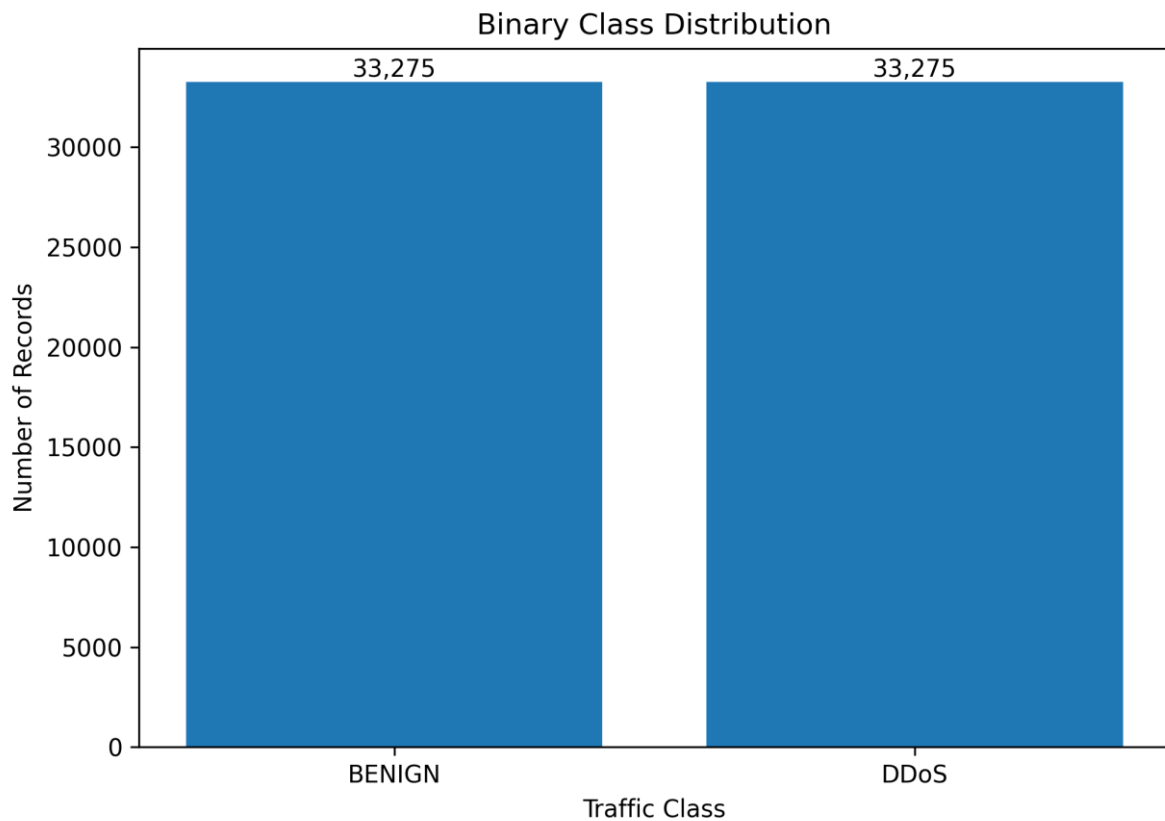


Figure 2. Actual project EDA: balanced binary class distribution after cleaning and attack sampling.

Figure 2 verifies the final binary balance: 33,275 BENIGN records and 33,275 DDoS records. This balance is a property of the prepared experimental dataset, not a claim about real-world traffic prevalence.

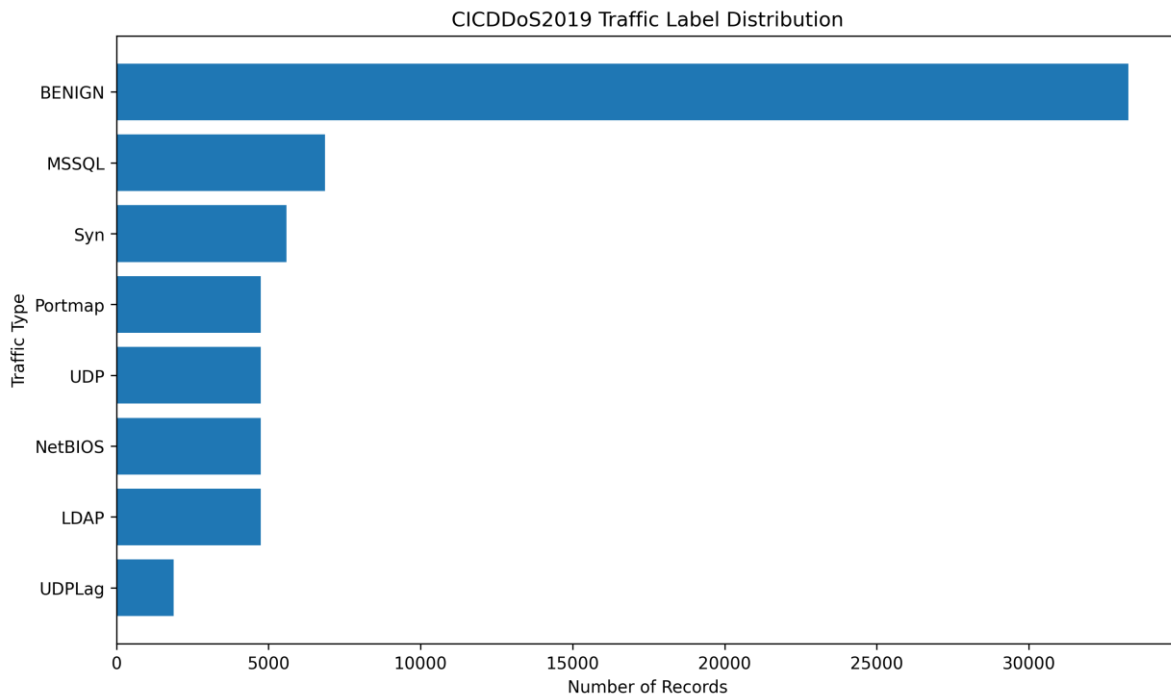


Figure 3. Actual project EDA: retained CICDDoS2019 traffic-label distribution.

The attack subclasses are not individually balanced. MSSQL is the largest retained attack family with 6,850 records, while UDPLag is the smallest with 1,873. The binary learning target groups all seven retained attack types into the DDoS class.

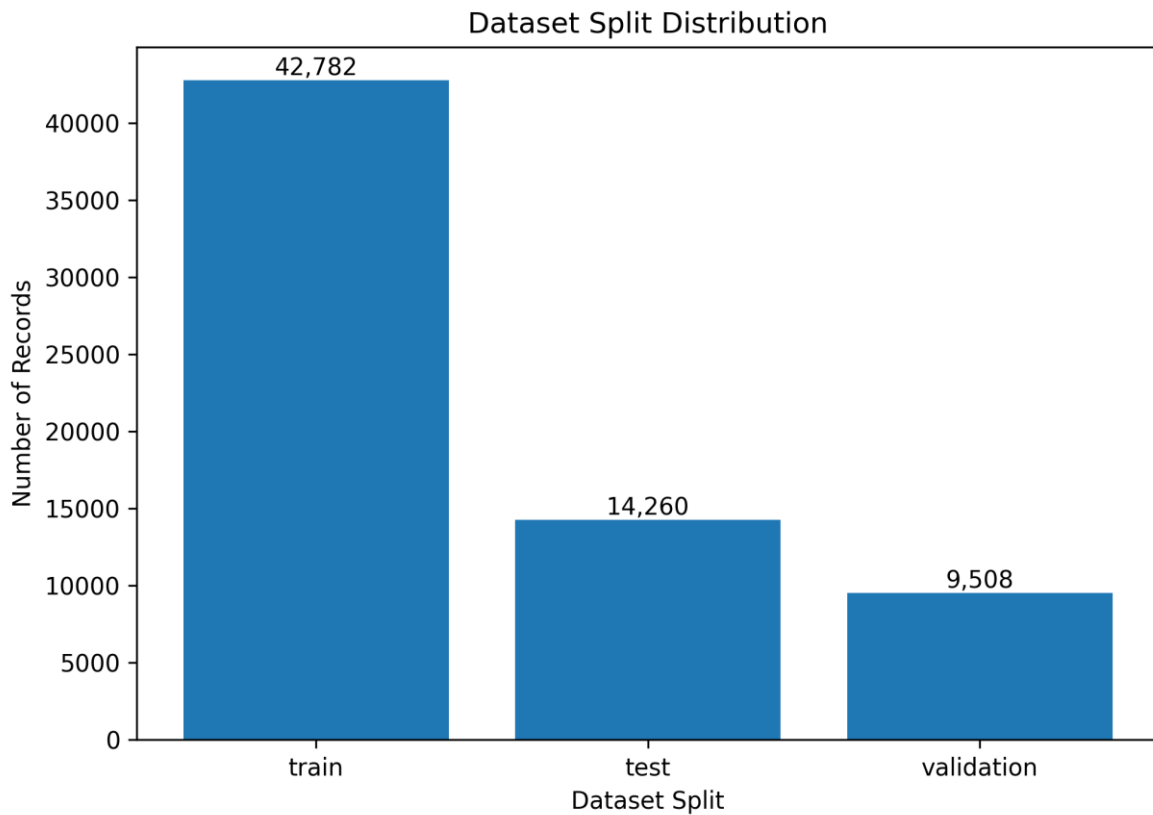


Figure 4. Actual project EDA: final dataset split sizes.

The final split consists of 42,782 training flows, 9,508 validation flows, and 14,260 test flows. Each split remains exactly balanced between BENIGN and DDoS for the binary task.

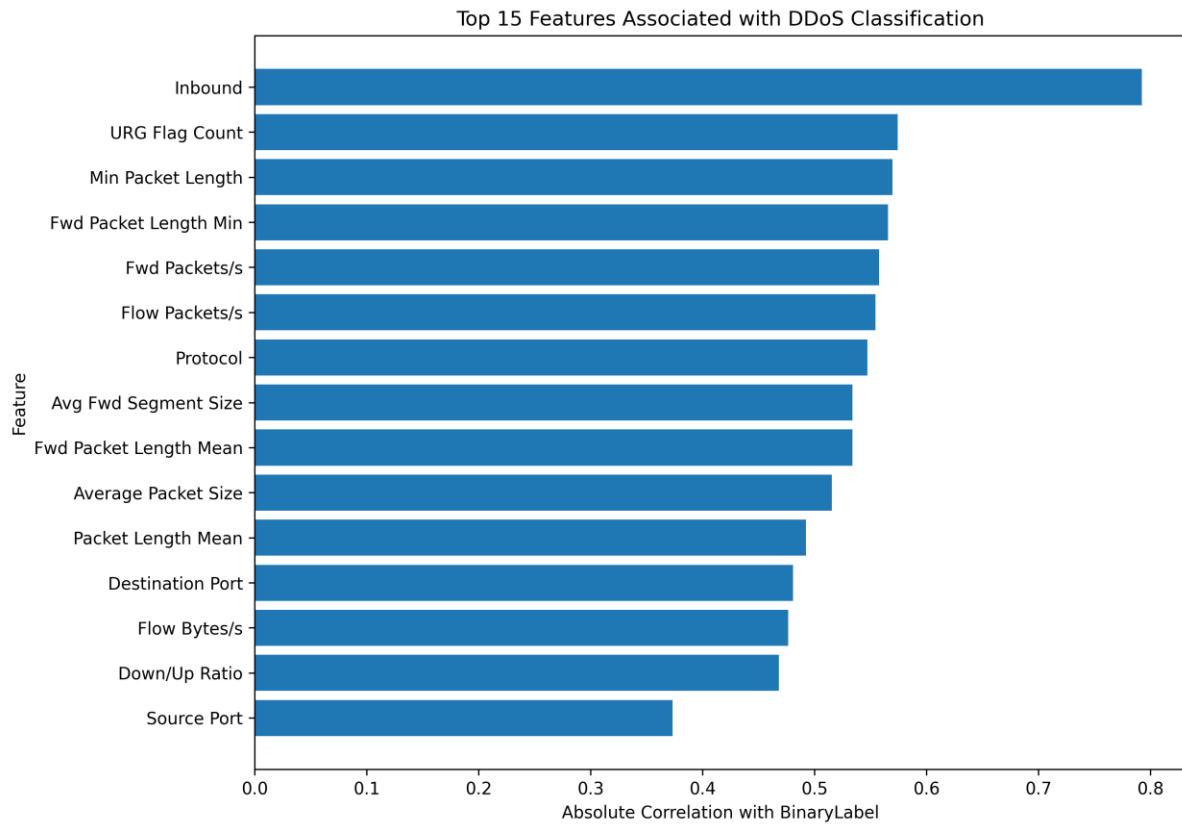


Figure 5. Actual project EDA: top 15 features by absolute correlation with BinaryLabel.

Table 7. Highest feature-target associations shown by the project correlation analysis

Feature	Absolute correlation with BinaryLabel
Inbound	0.792143
URG Flag Count	0.574130
Min Packet Length	0.569485
Fwd Packet Length Min	0.565572
Fwd Packets/s	0.557504
Flow Packets/s	0.554461
Protocol	0.547057
Avg Fwd Segment Size	0.533798
Fwd Packet Length Mean	0.533798
Average Packet Size	0.515514

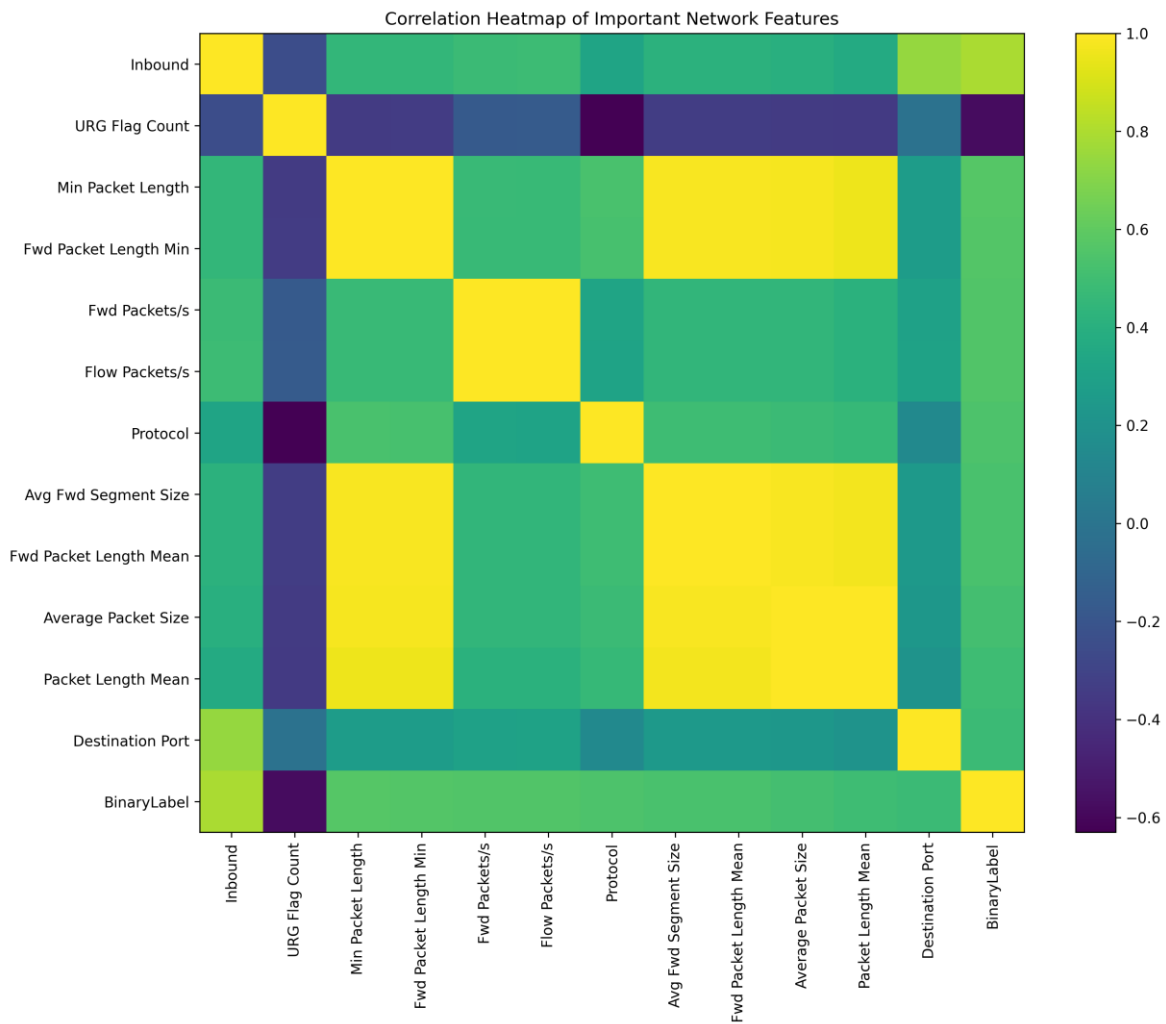


Figure 6. Actual project EDA: correlation heatmap for important network features and BinaryLabel.

The heatmap shows that several packet-length features are strongly correlated with one another, while their relationships with BinaryLabel are moderate rather than independent. This is expected in flow statistics because derived measures such as minimum, mean, segment size, and average packet size describe related aspects of the same traffic. The project does not remove all correlated features; instead, it preserves the defined 68-feature representation for the three model pipelines.

3.5 Summary of EDA Findings

- The final binary experiment is exactly balanced at 33,275 BENIGN and 33,275 DDoS flows.
- The retained DDoS family composition remains heterogeneous: MSSQL and Syn are more frequent than UDPLag, while LDAP, NetBIOS, UDP, and Portmap are close in retained count.
- The train/validation/test split is 42,782 / 9,508 / 14,260 flows and remains binary-balanced in every partition.
- Inbound is the strongest single absolute correlation with BinaryLabel among the reported top features (0.792143). Other strong associations include URG Flag Count, minimum packet-length measures, and packet-rate measures.

- Several packet-size and packet-rate features are mutually correlated, confirming redundancy in network-flow statistics but not invalidating their use in the defined 68-feature model input.
- Histograms and boxplots generated by the project show non-uniform feature distributions and substantial range differences, making consistent numerical preprocessing important before Logistic Regression, LSTM, and Autoencoder training.
- The EDA findings support a controlled binary case study; they do not establish that the same feature relationships or class proportions will hold on unrelated networks.

4. ALGORITHM SELECTION AND IMPLEMENTATION

4.1 Rationale for the Three-Model Design

The final project intentionally uses three models with different inductive assumptions. Logistic Regression provides a classical linear probability baseline, the LSTM adds sequential context, and the Autoencoder provides a behavior-based anomaly score learned only from BENIGN traffic. Using all three allows the case study to compare supervised per-flow classification, supervised temporal classification, and unsupervised reconstruction-based anomaly detection without introducing unrelated algorithms that were not implemented.

Table 8. Role of each implemented algorithm

Model	Learning mode	Evaluation unit	Implemented role
Logistic Regression	Supervised	Single flow	Strong classical baseline and per-flow DDoS probability
LSTM	Supervised deep learning	8-flow sequence	Learns ordered temporal dependencies across successive flows
Autoencoder	Unsupervised with respect to attack examples	Single flow	Learns BENIGN reconstruction and flags high-error anomalies

4.2 Common Input and Preprocessing Pipeline

All three models ultimately use the same 68 numerical flow features and the same StandardScaler fitted on training data. BinaryLabel is encoded as 0 for BENIGN and 1 for DDoS. Label and Split are retained for analysis and partitioning but are not predictive inputs. Flow ID, Source IP, Destination IP, Timestamp, SourceCSV, and SimillarHTTP are excluded from the model feature matrix.

For notebook training, infinite values are converted to missing values and the training-set medians are used to fill invalid numerical entries before scaling. The scaler is fitted only on X_train and then used to transform X_val and X_test. The fitted scaler and exact feature list are saved to backend/saved_models/ so that the deployed application uses the same feature ordering and transformation parameters.

For live CSV deployment, model_service.py validates the presence of all 68 features, converts them to numeric, replaces positive/negative infinity with missing values, and fills malformed or missing entries using the training means stored inside StandardScaler before applying scaler.transform(). This deployment fallback is explicitly documented because it is not identical to the notebook's median-imputation rule.

4.3 Logistic Regression

Logistic Regression was selected as the baseline because the binary target is well-defined, the model produces calibrated-style class probabilities, and it offers a computationally light reference against which the deep-learning models can be compared. The implementation uses scikit-learn LogisticRegression with max_iter=1000, class_weight="balanced", and n_jobs=-1.

For a scaled feature vector x , the model computes a linear score $z = w^T x + b$ and applies the sigmoid transformation $p = 1 / (1 + \exp(-z))$. During test evaluation and live inference, $p \geq 0.5$ is classified as DDoS. The trained model is saved as `logistic_regression.joblib`.

On the 14,260-flow test split, the model produced 7,122 true BENIGN predictions, 8 benign false alarms, 23 missed DDoS flows, and 7,107 correctly detected DDoS flows. This corresponds to 99.7826% accuracy and 99.6774% recall.

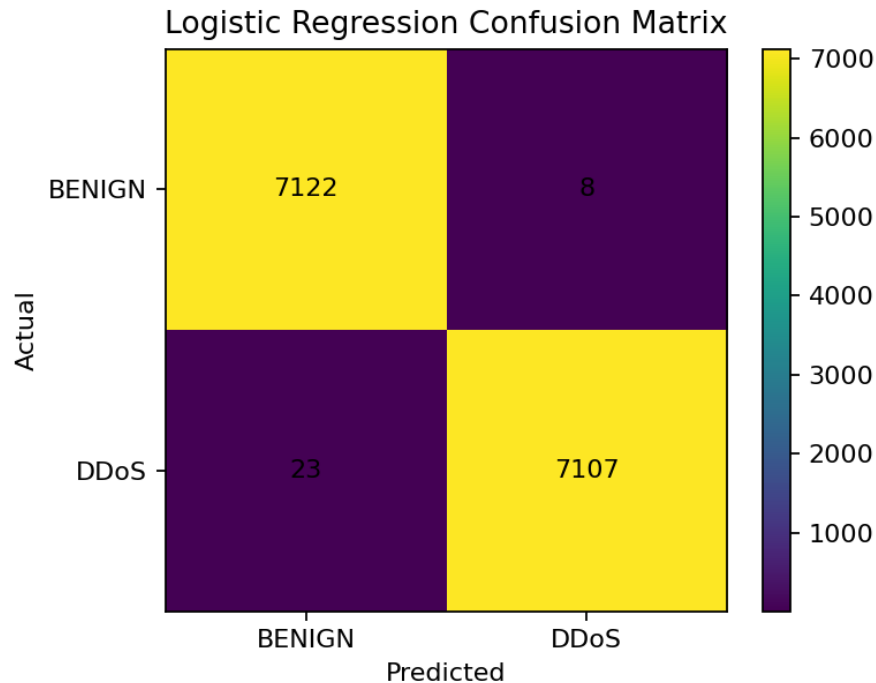


Figure 7. Actual project result: Logistic Regression confusion matrix on the 14,260-flow test set.

4.4 Temporal LSTM Model

The LSTM was selected because network flows can be ordered and evaluated as short sequences rather than as completely independent observations. The project uses a sequence length of eight. The clean temporal dataset is loaded so that Timestamp and SourceCSV remain available. Within each predefined split, rows are grouped by SourceCSV, sorted by Timestamp, and converted into overlapping windows. A sequence is never allowed to cross from one source CSV into another.

Each sequence therefore has shape (8, 68). The target is the BinaryLabel of the final flow in the eight-flow window. This process produces 42,733 training sequences, 9,459 validation sequences, and 14,211 test sequences. The difference from the original flow counts occurs because each source group loses up to seven initial positions before the first complete sequence can be formed.

The implemented neural architecture is: Input(8,68) -> LSTM(64, return_sequences=True) -> Dropout(0.30) -> LSTM(32) -> Dropout(0.20) -> Dense(16, ReLU) -> Dense(1, Sigmoid). The notebook records 47,009 total parameters. Training uses the Adam optimizer, binary cross-entropy loss, batch size 256, and a maximum of 15 epochs. EarlyStopping monitors validation

loss with patience 3 and restores the best weights; ModelCheckpoint saves the best validation-loss model as lstm_model.keras.

The LSTM test set contains 7,114 BENIGN sequence labels and 7,097 DDoS sequence labels. The final confusion matrix is $[[7109, 5], [15, 7082]]$, meaning 20 of 14,211 test sequences are misclassified. Because the data split was created before chronological ordering, these results must be described as chronological sequence modeling within existing splits, not as a strict future-time holdout experiment.

Table 9. Implemented LSTM architecture

Layer	Configuration
Input	8 time steps x 68 features
LSTM 1	64 units, return_sequences=True
Dropout 1	0.30
LSTM 2	32 units
Dropout 2	0.20
Dense	16 units, ReLU
Output	1 unit, Sigmoid
Total parameters	47,009

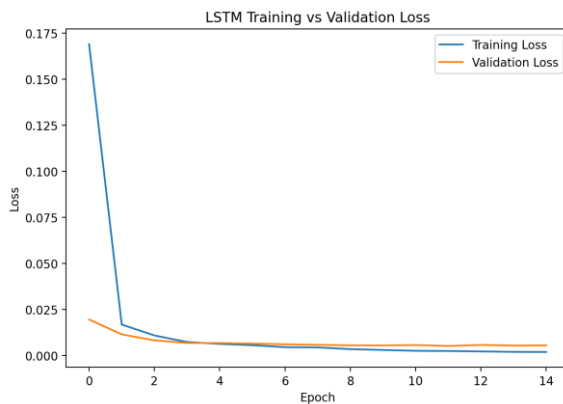


Figure 8(a). Actual LSTM training and validation loss.

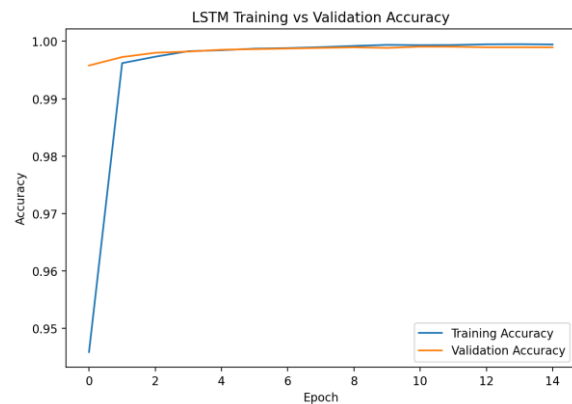


Figure 8(b). Actual LSTM training and validation accuracy.

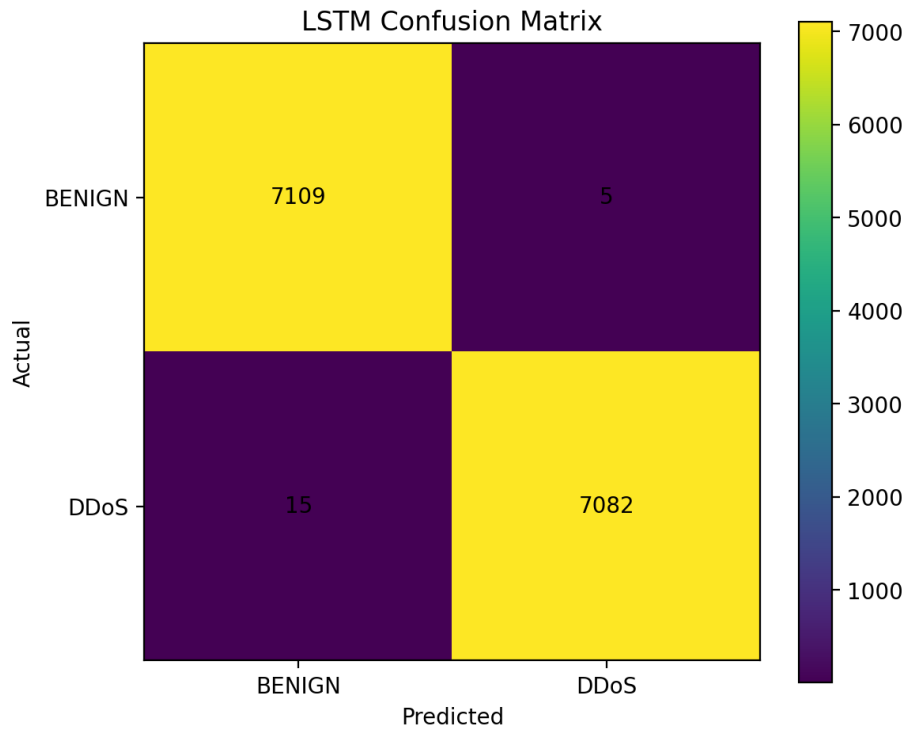


Figure 9. Actual project result: LSTM confusion matrix on 14,211 test sequences.

4.5 Autoencoder-Based Anomaly Detection

The Autoencoder was selected to provide an anomaly-detection component that does not learn attack classes directly. Only BENIGN training records are supplied as Autoencoder inputs and reconstruction targets. The model therefore learns a compressed representation of normal traffic and is expected to reconstruct familiar BENIGN patterns more accurately than traffic that differs from those patterns.

The implemented network uses 68 inputs followed by Dense(48, ReLU), Dropout(0.10), Dense(24, ReLU), a 12-unit ReLU latent space, Dense(24, ReLU), Dense(48, ReLU), and a 68-unit linear output layer. The notebook records 9,632 total parameters. The model is compiled with Adam and mean squared error, trained with batch size 256 for up to 30 epochs, and uses EarlyStopping with patience 4 plus ModelCheckpoint on validation loss. Only BENIGN validation samples are used for validation loss during reconstruction training.

After training, reconstruction error is calculated for the complete validation and test sets. The project's final threshold is selected from the validation precision-recall curve by maximizing validation F1, not by optimizing on the test labels. This produced $\tau = 0.126525490859982$ and validation F1 = 0.9100173043640488. The final test predictions are then produced once using $\text{error} > \tau$ as the DDoS-like anomaly decision.

With the final threshold, the Autoencoder confusion matrix is $[[5750, 1380], [28, 7102]]$. The model therefore misses only 28 of 7,130 DDoS test flows, yielding 99.6073% recall, but it also flags 1,380 BENIGN flows as anomalous. This explains why the Autoencoder has much lower accuracy and precision than the supervised models while still acting as a high-sensitivity detector.

Table 10. Implemented Autoencoder architecture

Stage	Units / Activation
Input	68 network-flow features
Encoder 1	48 / ReLU
Dropout	0.10
Encoder 2	24 / ReLU
Latent space	12 / ReLU
Decoder 1	24 / ReLU
Decoder 2	48 / ReLU
Output	68 / Linear
Total parameters	9,632

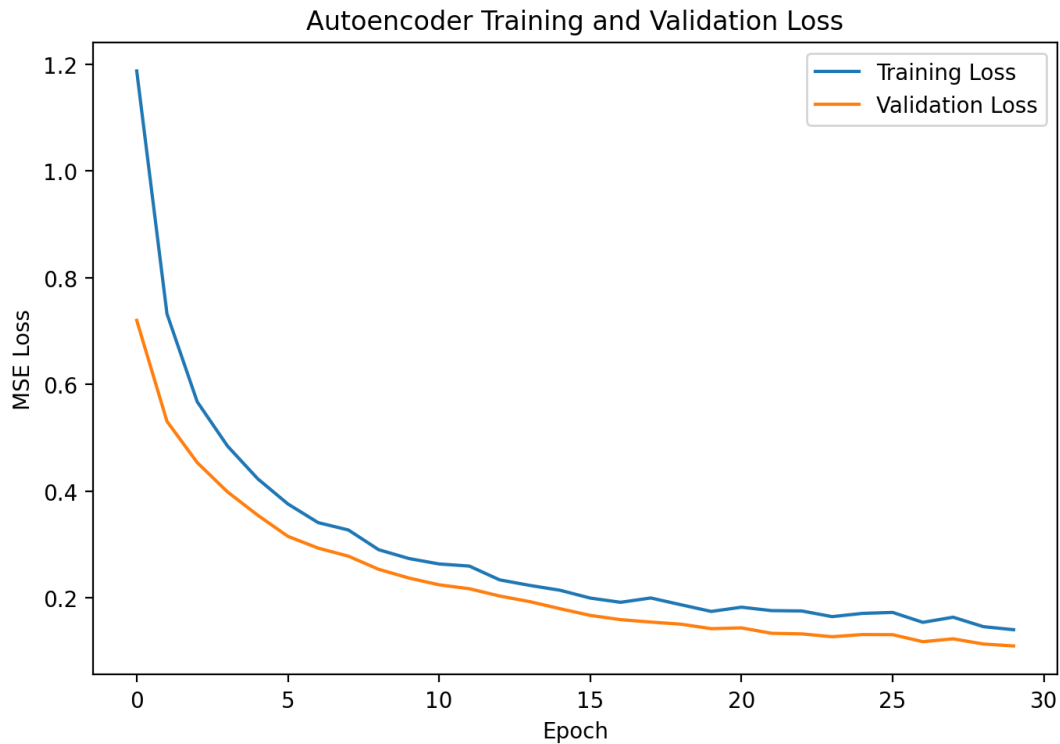


Figure 10. Actual project result: Autoencoder training and BENIGN validation reconstruction loss.

Table 11. Final optimized Autoencoder confusion matrix from the notebook output

Actual / Predicted	BENIGN	DDoS-like anomaly
BENIGN	5,750	1,380
DDoS	28	7,102

4.6 Evaluation Metrics Used

Accuracy measures the fraction of all evaluated units that are correctly classified. Precision measures the fraction of predicted DDoS units that are truly DDoS. Recall measures the fraction of true DDoS units that are detected. F1-score is the harmonic mean of precision and recall. ROC-AUC evaluates ranking across false-positive and true-positive rates, while PR-AUC emphasizes precision-recall behavior and is useful for attack-detection analysis.

In this project, recall deserves particular attention because a false negative represents DDoS traffic that the detector does not flag. Precision is also operationally important because excessive false positives can overwhelm an analyst. The Autoencoder demonstrates this trade-off directly: the final threshold produces very high DDoS recall at the cost of a larger benign false-positive count.

Evaluation formulas: Accuracy = $(TP + TN) / (TP + TN + FP + FN)$
Precision = $TP / (TP + FP)$
Recall = $TP / (TP + FN)$
F1 = $2 * Precision * Recall / (Precision + Recall)$

4.7 Final Results and Discussion

Table 12. Final measured project results

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	PR-AUC
Logistic Regression	99.7826%	99.8876%	99.6774%	99.7824%	99.9270%	99.9489%
Temporal LSTM	99.8593%	99.9294%	99.7886%	99.8590%	99.9902%	99.9916%
Autoencoder	90.1262%	83.7303%	99.6073%	90.9813%	91.4257%	86.0693%

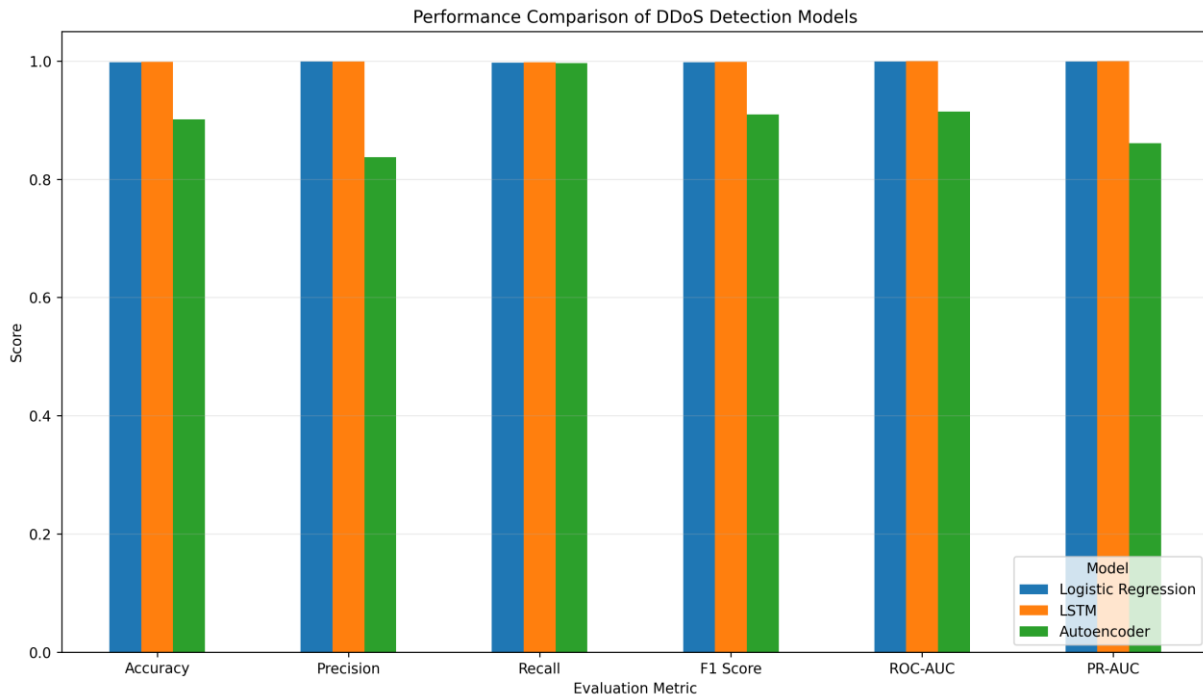


Figure 11. Actual project graph: final comparison of Logistic Regression, LSTM, and optimized Autoencoder results.

The Logistic Regression baseline is already highly effective on the balanced binary flow task. Its confusion matrix contains only 31 errors across 14,260 test flows. This shows that the selected numerical flow features make the binary separation relatively easy for a linear supervised model on this prepared subset.

The LSTM produces the strongest measured classification metrics in the final comparison, including 99.8593% accuracy, 99.9902% ROC-AUC, and 99.9916% PR-AUC. The absolute improvement over Logistic Regression is small, so the result should not be interpreted as evidence that recurrent modeling is universally necessary. It shows that, within this project setup, the sequence model performs at least as strongly while incorporating short-term ordered context.

The Autoencoder should not be judged by accuracy alone because it is trained under a different objective. It never receives DDoS examples during reconstruction training. The final validation-selected threshold deliberately favors attack sensitivity: only 28 DDoS flows are missed, but 1,380 BENIGN flows are flagged. This gives 99.6073% recall but 83.7303% precision. Its value in the project is therefore as a complementary anomaly detector rather than as a replacement for the supervised classifiers.

The comparison is also subject to a unit-of-analysis difference. Logistic Regression and the Autoencoder evaluate 14,260 individual test flows, whereas the LSTM evaluates 14,211 sequence targets. The table is useful for reporting the actual project results, but it should not be described as a perfectly identical head-to-head experiment.

4.8 Deployment Implementation

The final trained system is deployed through a Flask application in backend/app.py and backend/model_service.py. At startup, DDoSModelService reads feature_columns.json, model_config.json, autoencoder_threshold.json, and final_metrics.json, then loads the

StandardScaler, Logistic Regression model, LSTM model, and Autoencoder model. Keras models are loaded with compile=False because the deployed use case is inference-only.

The API implements GET /api/health, GET /api/metrics, GET /api/config, POST /api/validate, and POST /api/predict. /api/validate reads a small sample of the uploaded CSV and reports missing features. /api/predict reads the full CSV, applies preprocessing, executes the models that are available, and returns model summaries plus sample-level results and dynamic analytics. The Flask configuration limits a single CSV upload to 64 MB.

Logistic Regression predicts every flow using a probability threshold of 0.5. The Autoencoder reconstructs every scaled flow and compares its reconstruction error with the saved threshold. The LSTM forms eight-flow windows and associates each sequence prediction with its final row. If SourceCSV and Timestamp are present, groups are ordered chronologically; otherwise uploaded row order is used.

The frontend is implemented with React and Vite. After a successful upload, the dashboard can display the primary BENIGN/DDoS distribution, model attack-rate comparison, Logistic Regression-Autoencoder agreement, an Autoencoder reconstruction-error histogram, an LSTM sequence probability trend, and sample predictions. The research metrics remain fixed because unlabeled live uploads do not contain ground truth from which new accuracy, precision, recall, or F1 values could be validly computed.

Table 13. Implemented Flask REST API

HTTP method	Endpoint	Implemented purpose
GET	/api/health	Report artifact/model readiness and configuration status
GET	/api/metrics	Return stored held-out experiment metrics
GET	/api/config	Return feature count, required features, sequence length, and Autoencoder threshold
POST	/api/validate	Check whether an uploaded CSV contains all required model features
POST	/api/predict	Run preprocessing and available models, then return CSV-specific analytics

4.9 End-to-End Input-to-Output Trace

12. Input: a CSV containing the 68 required numerical model features; SourceCSV and Timestamp are optional deployment metadata for LSTM ordering.
13. Validation: column names are cleaned and the backend checks for missing required features.
14. Numerical preparation: infinities and non-numeric values are converted to missing values; deployment fallback values are taken from the scaler training means.
15. Scaling: the saved training-fitted StandardScaler transforms the 68-feature matrix.
16. Logistic Regression: per-flow DDoS probability and binary prediction are computed.
17. Autoencoder: per-flow reconstruction error is computed and compared with the saved anomaly threshold.

18. LSTM: ordered eight-flow windows are created and the probability for the final flow of each window is computed.
19. Aggregation: model attack counts, benign counts, rates, agreement, reconstruction-error histogram, and probability trends are constructed.
20. Output: the Flask API returns JSON that the React/Vite dashboard renders as live CSV analytics while keeping held-out research metrics separate.

5. CONCLUSION

This case-study report documents the SentinelFlow project exactly as implemented: a CICDDoS2019-based DDoS detection system that integrates data preparation, EDA, supervised machine learning, temporal deep learning, unsupervised anomaly detection, evaluation, model serialization, a Flask REST backend, and a React/Vite dashboard.

The data pipeline begins with a 70,000-row reduction of CICDDoS2019 CSV-03-11 and produces a 66,550-record balanced binary dataset after duplicate removal and controlled attack sampling. The model representation contains 68 numerical features. Identifier and raw metadata fields are excluded from direct prediction, constant features are removed, invalid values are treated using training-only statistics, and StandardScaler is fitted only on the training split. The separate clean temporal dataset retains Timestamp and SourceCSV so that the LSTM can construct ordered sequences without treating those fields as numerical predictors.

The Logistic Regression baseline demonstrates that the prepared binary task is strongly separable using the selected flow statistics. It achieves 99.7826% accuracy and 99.6774% recall. The LSTM incorporates eight-flow temporal context and achieves the strongest final supervised metrics: 99.8593% accuracy, 99.7886% recall, 99.9902% ROC-AUC, and 99.9916% PR-AUC. However, the numerical advantage over the baseline is small, and the LSTM's test unit is a sequence rather than an individual flow. These facts are important for an accurate interpretation of the result.

The Autoencoder contributes a different capability. It is trained only on BENIGN traffic and uses reconstruction error as an anomaly score. The final threshold of 0.126525490859982 is selected from validation precision-recall behavior rather than from the test set. At this threshold, the Autoencoder detects 7,102 of 7,130 DDoS test flows, giving 99.6073% recall, while also producing 1,380 benign false positives. This shows the expected sensitivity-specificity trade-off of anomaly detection and supports the project's decision to present the Autoencoder as a complementary detector rather than the single primary classifier.

The deployment layer successfully carries the trained experiment into a working interface. The Flask backend loads saved artifacts, validates input schemas, applies the scaler, runs the three models when available, and returns dynamic prediction analytics. The React/Vite dashboard displays those results while keeping them distinct from fixed held-out research metrics. This distinction is scientifically important because accuracy cannot be recomputed for an arbitrary unlabeled CSV.

The final project therefore meets its academic objective of comparing multiple machine-learning approaches for DDoS detection and demonstrating reproducible deployment. Its conclusions remain bounded by the actual experiment: the data is a balanced subset from one CICDDoS2019 project source, the target is binary, the LSTM is chronological only within predefined splits, and the application performs batch CSV analysis rather than live packet inspection. Future work could evaluate strict time-based or day-based holdouts, multiclass attack-family prediction, realistic class imbalance, cross-dataset generalization, and streaming flow ingestion, but those extensions are not claimed as part of the current implementation.



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

SIGNATURE OF THE TEAM MEMBERS

We confirm that this case-study report represents the project implementation and experimental results described in the submitted SentinelFlow project materials. Any remaining administrative detail marked as [TO BE CONFIRMED] should be verified before final submission.

Tej Pratap Singh

Roll No.: 2520030476

B. Aryan

Roll No.: 2520030343

Guide Signature: _____

Ms. Jyothsna Datti